

Research Notes on Support vector machine

Support vector machine

>> Section 1

$$D = \{ \mathbf{x}_i \mid \mathbf{x}_i \in \mathbb{R}^p \}_{i=1}^k \quad \{\displaystyle \mathcal{D}\}^{\star} = \{ \mathbf{x}_{\star} \mid \mathbf{x}_{\star} \in \mathbb{R}^p \}$$

>> Section 2

Under certain assumptions about the sequence of random variables X_k, y_k (for example, that they are generated by a finite Markov process), if the set of hypotheses being considered is small enough, the minimizer of the empirical risk will closely approximate the minimizer of the expected risk as n grows large. This approach is called empirical risk minimization, or ERM.

>> Section 3

This approach is called Tikhonov regularization. More generally, $R(f)$ can be some measure of the complexity of the hypothesis f , so that simpler hypotheses are preferred.

>> Section 4

Target functions The difference between the hinge loss and these other loss functions is best stated in terms of target functions - the function that minimizes expected risk for a given pair of random variables X, y . In particular, let y_x denote y conditional on the event that $X = x$. In the classification setting, we have:

>> Section 5

where the parameter $C > 0$ determines the trade-off between increasing the margin size and ensuring that the $\mathbf{x}_i$ lie on the correct side of the margin (Note we can add a weight to either term in the equation above). By deconstructing the hinge loss, this optimization problem can be formulated into the following:

>> Section 6

maximize $f(c_1, \dots, c_n) = \sum_{i=1}^n c_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n y_i c_i (x_i^T x_j) y_j c_j$, subject to $\sum_{i=1}^n c_i y_i = 0$, and $0 \leq c_i \leq \frac{1}{2n\lambda}$ for all i .

$$\begin{aligned} c_n &= \sum_{i=1}^n c_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n y_i c_i (x_i^T x_j) y_j c_j \\ &\quad \text{subject to } \sum_{i=1}^n c_i y_i = 0, \text{ and } 0 \leq c_i \leq \frac{1}{2n\lambda} \text{ for all } i. \end{aligned}$$

>> Section 7

The original maximum-margin hyperplane algorithm proposed by Vapnik in 1963 constructed a linear classifier. However, in 1992, Bernhard Boser, Isabelle Guyon and Vladimir Vapnik suggested a way to create nonlinear classifiers by applying the kernel trick (originally proposed by Aizerman et al.) to maximum-margin hyperplanes. The kernel trick, where dot products are replaced by kernels, is easily derived in the dual representation of the SVM problem. This allows the algorithm to fit the maximum-margin hyperplane in a transformed feature space. The transformation may be nonlinear and the transformed space high-dimensional; although the classifier is a hyperplane in the transformed feature space, it may be nonlinear in the original input space. It is noteworthy that working in a higher-dimensional feature space increases the generalization error of support vector machines, although given enough samples the algorithm still performs well. Some common kernels include:

>> Section 8

where x_i is a training sample with target value y_i . The inner product plus intercept $w^T x_i + b$ is the prediction for that sample, and ϵ is a free parameter that serves as a threshold: all predictions have to be within an ϵ range of the true predictions. Slack variables are usually added into the above to allow for errors and to allow approximation in the case the above problem is infeasible.

>> Section 9

$$b = w^T \phi(x_i) - y_i = \left[\sum_{j=1}^n c_j y_j \phi(x_j) \cdot \phi(x_i) \right] - y_i = \left[\sum_{j=1}^n c_j y_j k(x_j, x_i) \right] - y_i$$

>> Section 10

Classifying data is a common task in machine learning. Suppose some given data points each belong to one of two classes, and the goal is to decide which class a new data point will be in. In the case of support vector machines, a data point is viewed as a p -dimensional vector (a list of p numbers), and we want to know whether we can separate such points with a $(p-1)$ -dimensional hyperplane. This is called a linear classifier. There are many hyperplanes that might classify the data. One reasonable choice as the best hyperplane is the one that

Research Notes on Support vector machine

Support vector machine

represents the largest separation, or margin, between the two classes. So we choose the hyperplane so that the distance from it to the nearest data point on each side is maximized. If such a hyperplane exists, it is known as the maximum-margin hyperplane and the linear classifier it defines is known as a maximum-margin classifier; or equivalently, the perceptron of optimal stability. More formally, a support vector machine constructs a hyperplane or set of hyperplanes in a high or infinite-dimensional space, which can be used for classification, regression, or other tasks like outlier detection. Intuitively, a good separation is achieved by the hyperplane that has the largest distance to the nearest training-data point of any class (so-called functional margin), since in general the larger the margin, the lower the generalization error of the classifier. A lower generalization error means that the implementer is less likely to experience overfitting. Whereas the original problem may be stated in a finite-dimensional space, it often happens that the sets to discriminate are not linearly separable in that space. For this reason, it was proposed that the original finite-dimensional space be mapped into a much higher-dimensional space, presumably making the separation easier in that space. To keep the computational load reasonable, the mappings used by SVM schemes are designed to ensure that dot products of pairs of input data vectors may be computed easily in terms of the variables in the original space, by defining them in terms of a kernel function $k(x, y)$ selected to suit the problem. The hyperplanes in the higher-dimensional space are defined as the set of points whose dot product with a vector in that space is constant, where such a set of vectors is an orthogonal (and thus minimal) set of vectors that defines a hyperplane. The vectors defining the hyperplanes can be chosen to be linear combinations with parameters α_i of images of feature vectors x_i that occur in the data base. With this choice of a hyperplane, the points x in the feature space that are mapped into the hyperplane are defined by the relation $\sum_i \alpha_i k(x_i, x) = \text{constant}$. Note that if $k(x, y)$ becomes small as y grows further away from x , each term in the sum measures the degree of closeness of the test point x to the corresponding data base point x_i . In this way, the sum of kernels above can be used to measure the relative nearness of each test point to the data points originating in one or the other of the sets to be discriminated. Note the fact that the set of points x mapped into any hyperplane can be quite convoluted as a result, allowing much more complex discrimination between sets

that are not convex at all in the original space.